

KUBERNETES (K8S) TASK - 06

1.Create a Config Map from a directory containing multiple files and inject the variables into a pod as environment variables.

Prerequisites

Make sure your cluster is running:

```
kubectl get nodes
```

You should see something similar to:

NAME	STATUS	ROLES	AGE
k8s-master	Ready	control-plane	
k8s-worker1	Ready	<none>	
k8s-worker2	Ready	<none>	

Also verify:

```
kubectl get pods
```

ConfigMap from a directory → Environment Variables

Step 1: Create a directory

On your Kubernetes master:

```
mkdir configmap-demo
```

```
cd configmap-demo
```

Step 2: Create multiple configuration files

Create the first file:

```
vi app.properties
```

```
ubuntu@k8s-master: ~/configmap-demo
APP_NAME=MyApplication
APP_ENV=development
APP_PORT=8080
~
~
~
~
~
~
~
~
```

Create another file:

vi database.properties

```
ubuntu@k8s-master: ~/configmap-demo
GNU nano 8.7.1 database
DB_HOST=mysql
DB_PORT=3306
DB_NAME=mydatabase|
```

Check:

ls

You should see:

app.properties

database.properties

```
ubuntu@k8s-master: ~/configmap-demo
ubuntu@k8s-master:~/configmap-demo$ nano database.properties
ubuntu@k8s-master:~/configmap-demo$
ubuntu@k8s-master:~/configmap-demo$ ll
total 16
drwxrwxr-x 2 ubuntu ubuntu 4096 Sep 28 13:35 ./
drwxr-x--- 7 ubuntu ubuntu 4096 Sep 28 13:34 ../
-rw-rw-r-- 1 ubuntu ubuntu  57 Sep 28 13:34 app.properties
-rw-rw-r-- 1 ubuntu ubuntu  46 Sep 28 13:35 database.properties
ubuntu@k8s-master:~/configmap-demo$
```

Step 3: Create ConfigMap from the directory

Run:

```
kubectl create configmap app-config --from-file=configmap-demo/
```

If you are already inside configmap-demo, use:

```
kubectl create configmap app-config --from-file=.
```

Verify:

```
kubectl get configmap
```

You should see:

NAME	DATA
------	------

app-config	2
------------	---

```
ubuntu@k8s-master: ~/configmap-demo
ubuntu@k8s-master:~/configmap-demo$ kubectl create configmap app-config --from-f
ile=.
configmap/app-config created
ubuntu@k8s-master:~/configmap-demo$ kubectl get configmap
NAME          DATA  AGE
app-config    2      24s
kube-root-ca.crt 1      6d3h
ubuntu@k8s-master:~/configmap-demo$
```

Step 4: View ConfigMap

kubectl describe configmap app-config

You should see both files:

app.properties

database.properties

You can also run:

kubectl get configmap app-config -o yaml

```
ubuntu@k8s-master: ~/configmap-demo
ubuntu@k8s-master:~/configmap-demo$ kubectl describe configmap app-config
Name:         app-config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
app.properties:
-----
APP_NAME=MyApplication
APP_ENV=development
APP_PORT=8080

database.properties:
-----
DB_HOST=mysql
DB_PORT=3306
DB_NAME=mydatabase

BinaryData
====
Events:      <none>
ubuntu@k8s-master:~/configmap-demo$ kubectl get configmap app-config -o yaml
apiVersion: v1
data:
  app.properties: |
    APP_NAME=MyApplication
    APP_ENV=development
    APP_PORT=8080
  database.properties: |
    DB_HOST=mysql
    DB_PORT=3306
    DB_NAME=mydatabase
kind: ConfigMap
metadata:
  creationTimestamp: "2026-09-28T13:37:26Z"
  name: app-config
  namespace: default
  resourceVersion: "66119"
  uid: a9cbf484-cbdc-433f-aa15-0e146d77e778
ubuntu@k8s-master:~/configmap-demo$
```

Step 5: Create Pod YAML

Go back:

cd ..

Create:

vi configmap-env-pod.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 configmap-e  
apiVersion: v1  
kind: Pod  
metadata:  
  name: configmap-env-pod  
spec:  
  containers:  
  - name: nginx  
    image: nginx  
    envFrom:  
    - configMapRef:  
      name: app-config
```

Step 6: Create the Pod

kubectl apply -f configmap-env-pod.yaml

Check:

kubectl get pod configmap-env-pod

Expected:

configmap-env-pod 1/1 Running

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano configmap-env-pod.yaml  
ubuntu@k8s-master:~$ kubectl apply -f configmap-env-pod.yaml  
pod/configmap-env-pod created  
ubuntu@k8s-master:~$ kubectl get pod configmap-env-pod  
NAME                READY   STATUS    RESTARTS   AGE  
configmap-env-pod   1/1     Running   0           9s  
ubuntu@k8s-master:~$
```

Step 7: Verify environment variables

Run:

kubectl exec configmap-env-pod -- env

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl exec configmap-env-pod -- env  
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin  
HOSTNAME=configmap-env-pod  
NGINX_VERSION=1.31.6  
NJS_VERSION=1.0.1  
NJS_RELEASE=1~trixie  
ACME_VERSION=0.4.1  
PKG_RELEASE=1~trixie  
DYNPKG_RELEASE=1~trixie  
app.properties=APP_NAME=MyApplication  
APP_ENV=development  
APP_PORT=8080  
  
database.properties=DB_HOST=mysql  
DB_PORT=3306  
DB_NAME=mydatabase  
  
NGINX_SERVICE_PORT_80_TCP_PORT=80  
APACHE_SERVICE_PORT_80_TCP_ADDR=10.98.222.38  
KUBERNETES_SERVICE_PORT=443  
NGINX_LOADBALANCE_PORT=tcp://10.103.212.154:80
```

Important

envFrom takes the **keys from the ConfigMap** and creates environment variables.

2.Create a ConfigMap from a file and mount it as a volume inside a pod, ensuring the configuration data is available as files.

ConfigMap from a file → Mount as Volume

Now we will take one configuration file and make it available **inside the Pod as a file**.

Step 1: Create configuration file

vi nginx-custom.conf

Add:

```
server {  
  
    listen 8080;  
  
    server_name example.com;  
  
}
```

Save:

Esc

:wq

```
ubuntu@k8s-master: ~  
server{  
    listen 8080;  
    server_name example.com;  
}  
~  
~  
~  
~  
~  
~
```

Step 2: Create ConfigMap from the file

kubectl create configmap nginx-config --from-file=nginx-custom.conf

Verify:

kubectl get configmap nginx-config

Describe:

kubectl describe configmap nginx-config

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl create configmap nginx-config --from-file=nginx-custom.conf  
configmap/nginx-config created  
ubuntu@k8s-master:~$ kubectl get configmap nginx-config  
NAME          DATA   AGE  
nginx-config  1       7s  
ubuntu@k8s-master:~$ kubectl describe configmap nginx-config  
Name:         nginx-config  
Namespace:    default  
Labels:        <none>  
Annotations:   <none>  
  
Data  
====  
nginx-custom.conf:  
-----  
server{  
    listen 8080;  
    server_name example.com;  
}  
  
BinaryData  
====  
Events:        <none>  
ubuntu@k8s-master:~$
```

Step 3: Create Pod YAML

vi configmap-volume-pod.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 configmap-volume-pod.yaml  
apiVersion: v1  
kind: Pod  
metadata:  
  name: configmap-volume-pod  
spec:  
  containers:  
  - name: nginx  
    image: nginx  
    volumeMounts:  
    - name: config-volume  
      mountPath: /etc/myconfig  
  volumes:  
  - name: config-volume  
    configMap:  
      name: nginx-config
```

Step 4: Create Pod

kubectl apply -f configmap-volume-pod.yaml

Check:

kubectl get pod configmap-volume-pod

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f configmap-volume-pod.yaml  
pod/configmap-volume-pod created  
ubuntu@k8s-master:~$ kubectl get pod configmap-volume-pod  
NAME                READY   STATUS    RESTARTS   AGE  
configmap-volume-pod 1/1     Running   0           8s  
ubuntu@k8s-master:~$
```

Step 5: Check the mounted file

Run:

kubectl exec configmap-volume-pod -- ls -l /etc/myconfig

You should see:

nginx-custom.conf

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl exec configmap-volume-pod -- ls -l /etc/myconfig  
total 0  
lrwxrwxrwx 1 root root 24 Sep 28 14:11 nginx-custom.conf -> ../data/nginx-custom.conf  
ubuntu@k8s-master:~$ |
```

Step 6: Read the configuration

`kubectl exec configmap-volume-pod -- cat /etc/myconfig/nginx-custom.conf`

Expected:

```
server {  
    listen 8080;  
    server_name example.com;  
}
```

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl exec configmap-volume-pod -- cat /etc/myconfig/nginx-custom.conf  
server{  
    listen 8080;  
    server_name example.com;  
}  
ubuntu@k8s-master:~$ |
```

What happened?

The ConfigMap:

`nginx-custom.conf`

became a file inside the container:

`/etc/myconfig/nginx-custom.conf`

3.Create a Secret with sensitive information (username and password) and inject it into a pod as environment variables.

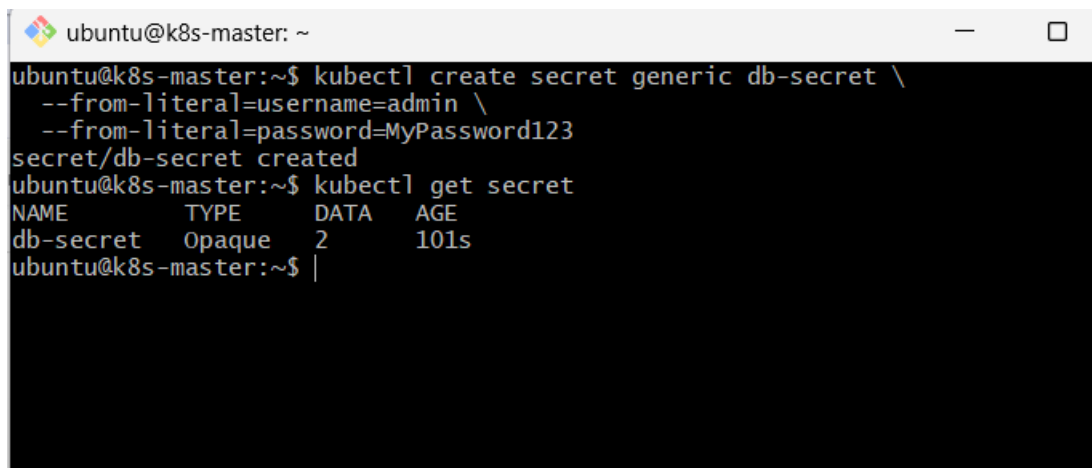
Secret → Environment Variables

Now we will create a Secret containing:

- username
- password

Step 1: Create Secret

You can create it directly from the command line:

A terminal window titled 'ubuntu@k8s-master: ~' showing the execution of two kubectl commands. The first command creates a secret named 'db-secret' with two keys: 'username' with value 'admin' and 'password' with value 'MyPassword123'. The second command lists the secret, showing its type as 'Opaque' and its age as '101s'.

```
ubuntu@k8s-master:~$ kubectl create secret generic db-secret \
--from-literal=username=admin \
--from-literal=password=MyPassword123
secret/db-secret created
ubuntu@k8s-master:~$ kubectl get secret
NAME          TYPE          DATA   AGE
db-secret     Opaque        2       101s
ubuntu@k8s-master:~$
```

Step 2: Describe Secret

kubectl describe secret db-secret

You will see the keys:

username

password

The actual values aren't displayed by describe.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl describe secret db-secret  
Name:          db-secret  
Namespace:     default  
Labels:        <none>  
Annotations:   <none>  
  
Type: Opaque  
  
Data  
====  
password: 13 bytes  
username: 5 bytes  
ubuntu@k8s-master:~$ |
```

Step 3: Create Pod YAML

vi secret-env-pod.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 secret-env-pod.yaml  
apiVersion: v1  
kind: Pod  
metadata:  
  name: secret-env-pod  
spec:  
  containers:  
  - name: nginx  
    image: nginx  
    env:  
    - name: DB_USERNAME  
      valueFrom:  
        secretKeyRef:  
          name: db-secret  
          key: username  
    - name: DB_PASSWORD  
      valueFrom:  
        secretKeyRef:  
          name: db-secret  
          key: password
```

Step 4: Create Pod

kubectl apply -f secret-env-pod.yaml

Check:

kubectl get pod secret-env-pod

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f secret-env-pod.yaml  
pod/secret-env-pod created  
ubuntu@k8s-master:~$ kubectl get pod secret-env-pod  
NAME          READY   STATUS    RESTARTS   AGE  
secret-env-pod 1/1     Running   0          24s  
ubuntu@k8s-master:~$
```

Step 5: Verify environment variables

`kubectl exec secret-env-pod -- env | grep DB_`

Expected:

`DB_USERNAME=admin`

`DB_PASSWORD=MyPassword123`

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl exec secret-env-pod -- env | grep DB_  
DB_USERNAME=admin  
DB_PASSWORD=MyPassword123  
ubuntu@k8s-master:~$
```

Important

For a real production environment, avoid exposing Secret values in terminal output, logs, shell history, or manifests unnecessarily. Kubernetes Secrets provide encoding and API-level controls; **base64 is not encryption by itself**.

4. Create a Secret using a YAML file, mount it as a volume in a pod, and verify the specific Secret values are available as files.

Secret YAML → Mount as Files

Now we'll create the Secret using a YAML file and mount it inside a Pod.

Step 1: Generate base64 values

Kubernetes Secret YAML normally stores data values as base64.

Run:

```
echo -n 'admin' | base64
```

Output:

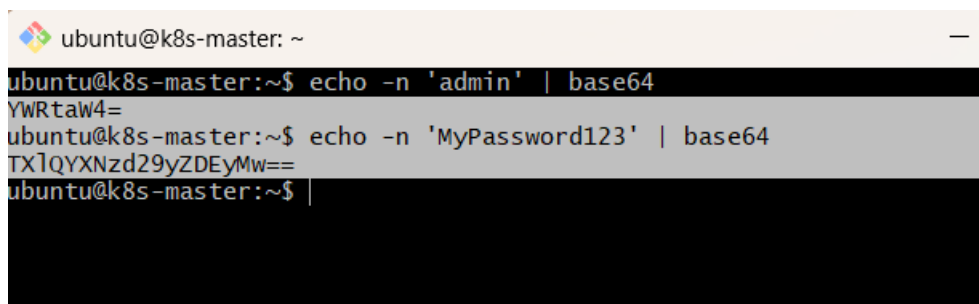
```
YWRtaW4=
```

Password:

```
echo -n 'MyPassword123' | base64
```

Output:

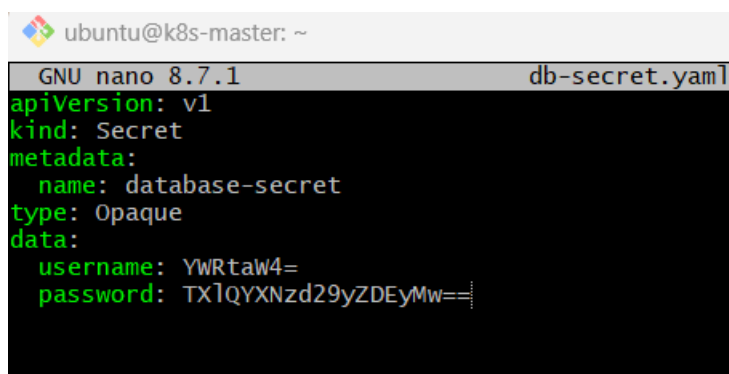
```
TXlQYXNzd29yZDEyMw==
```

A terminal window titled 'ubuntu@k8s-master: ~' showing two commands and their outputs. The first command is 'echo -n 'admin' | base64' which outputs 'YWRtaW4='. The second command is 'echo -n 'MyPassword123' | base64' which outputs 'TXlQYXNzd29yZDEyMw=='.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ echo -n 'admin' | base64  
YWRtaW4=  
ubuntu@k8s-master:~$ echo -n 'MyPassword123' | base64  
TXlQYXNzd29yZDEyMw==  
ubuntu@k8s-master:~$ |
```

Step 2: Create Secret YAML

vi db-secret.yaml

A terminal window titled 'ubuntu@k8s-master: ~' showing the nano editor editing 'db-secret.yaml'. The content of the file is: 'apiVersion: v1', 'kind: Secret', 'metadata:', ' name: database-secret', 'type: Opaque', 'data:', ' username: YWRtaW4=', ' password: TXlQYXNzd29yZDEyMw=='.

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 db-secret.yaml  
apiVersion: v1  
kind: Secret  
metadata:  
  name: database-secret  
type: Opaque  
data:  
  username: YWRtaW4=  
  password: TXlQYXNzd29yZDEyMw==
```

Step 3: Create Secret

kubectl apply -f db-secret.yaml

Verify:

kubectl get secret database-secret

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f db-secret.yaml  
secret/database-secret created  
ubuntu@k8s-master:~$ kubectl get secret database-secret  
NAME          TYPE      DATA   AGE  
database-secret  Opaque    2       8s  
ubuntu@k8s-master:~$
```

Step 4: Create Pod YAML

vi secret-volume-pod.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1      secret-volume-pod.yaml  
apiVersion: v1  
kind: Pod  
metadata:  
  name: secret-volume-pod  
spec:  
  containers:  
  - name: nginx  
    image: nginx  
    volumeMounts:  
    - name: secret-volume  
      mountPath: /etc/database-secret  
      readOnly: true  
  volumes:  
  - name: secret-volume  
    secret:  
      secretName: database-secret
```

Step 5: Create Pod

kubectl apply -f secret-volume-pod.yaml

Check:

kubectl get pod secret-volume-pod

Step 6: Check Secret files

kubectl exec secret-volume-pod -- ls -l /etc/database-secret

Expected:

password

username

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f secret-volume-pod.yaml  
pod/secret-volume-pod created  
ubuntu@k8s-master:~$ kubectl get pod secret-volume-pod  
NAME                READY   STATUS    RESTARTS   AGE  
secret-volume-pod   1/1     Running   0           6s  
ubuntu@k8s-master:~$ kubectl exec secret-volume-pod -- ls -l /etc/database-secre  
t  
total 0  
lrwxrwxrwx 1 root root 15 Sep 28 14:37 password -> ../data/password  
lrwxrwxrwx 1 root root 15 Sep 28 14:37 username -> ../data/username  
ubuntu@k8s-master:~$ |
```

Step 7: Verify username

```
kubectl exec secret-volume-pod -- cat /etc/database-secret/username
```

Output:

admin

Step 8: Verify password

```
kubectl exec secret-volume-pod -- cat /etc/database-secret/password
```

Output:

MyPassword123

So the Secret has been mounted like this:

/etc/database-secret/

├── username

└── password

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl exec secret-volume-pod -- cat /etc/database-secret/  
username  
ubuntu@k8s-master:~$ kubectl exec secret-volume-pod -- cat /etc/database-secret/  
usernamername  
ubuntu@k8s-master:~$ kubectl exec secret-volume-pod -- cat /etc/database-secret/  
passwordsword  
MyPassword123ubuntu@k8s-master:~$ |
```

5.Inject a ConfigMap as environment variables and a Secret as files into the same pod, ensuring both are accessible within the pod.

ConfigMap as Environment Variables + Secret as Files

This is an important real-world scenario.

We will use:

ConfigMap

APP_NAME

APP_ENV

APP_PORT

as environment variables.

And:

Secret

username

password

as files.

Step 1: Create ConfigMap

Let's create a dedicated ConfigMap:

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl create configmap application-config \  
  --from-literal=APP_NAME=MyApplication \  
  --from-literal=APP_ENV=production \  
  --from-literal=APP_PORT=8080  
configmap/application-config created  
ubuntu@k8s-master:~$ kubectl get configmap application-config  
NAME          DATA  AGE  
application-config  3      12s  
ubuntu@k8s-master:~$ |
```

Step 2: Create Secret

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl create secret generic application-secret \  
  --from-literal=username=admin \  
  --from-literal=password=MyPassword123  
secret/application-secret created  
ubuntu@k8s-master:~$ kubectl get secret application-secret  
NAME          TYPE      DATA  AGE  
application-secret  Opaque    2      25s  
ubuntu@k8s-master:~$ |
```

Step 3: Create combined Pod YAML

vi combined-pod.yaml

Add:

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: v1  
kind: Pod  
metadata:  
  name: combined-config-secret-pod  
spec:  
  containers:  
  - name: nginx  
    image: nginx  
  
    envFrom:  
    - configMapRef:  
      name: application-config  
  
    volumeMounts:  
    - name: secret-volume  
      mountPath: /etc/app-secret  
      readOnly: true  
  
  volumes:  
  - name: secret-volume  
    secret:  
      secretName: application-secret
```

Step 4: Create Pod

kubectl apply -f combined-pod.yaml

Check:

kubectl get pod combined-config-secret-pod

Expected:

combined-config-secret-pod 1/1 Running

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f combined-pod.yaml  
pod/combined-config-secret-pod created  
ubuntu@k8s-master:~$ kubectl get pod combined-config-secret-pod  
NAME                                READY   STATUS    RESTARTS   AGE  
combined-config-secret-pod         1/1     Running   0           6s  
ubuntu@k8s-master:~$ |
```

Step 5: Verify ConfigMap environment variables

Run:

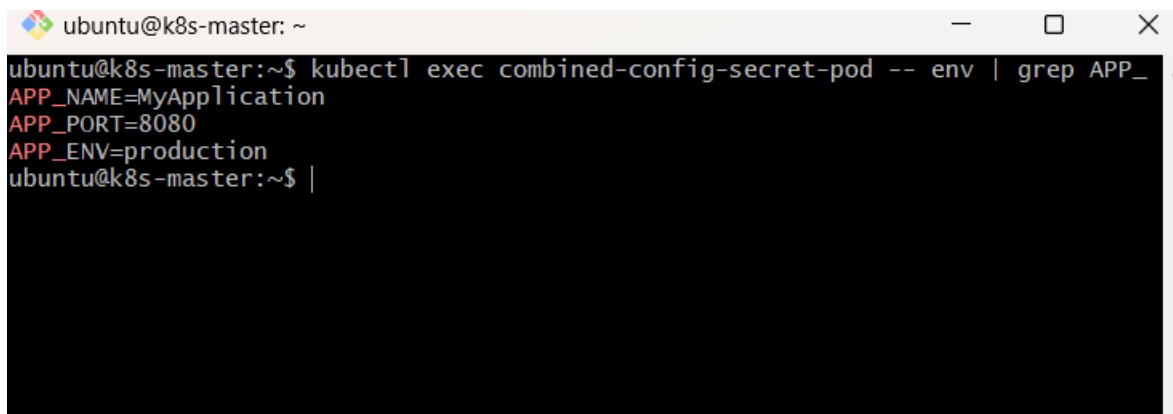
```
kubectl exec combined-config-secret-pod -- env | grep APP_
```

Expected:

APP_NAME=MyApplication

APP_ENV=production

APP_PORT=8080

A terminal window titled 'ubuntu@k8s-master: ~' with standard window controls. The command 'kubectl exec combined-config-secret-pod -- env | grep APP_' is entered. The output shows three lines: 'APP_NAME=MyApplication', 'APP_PORT=8080', and 'APP_ENV=production'. The prompt returns to 'ubuntu@k8s-master:~\$'.

Step 6: Verify Secret files

Check:

```
kubectl exec combined-config-secret-pod -- ls -l /etc/app-secret
```

Expected:

password

username

Check username:

```
kubectl exec combined-config-secret-pod -- cat /etc/app-secret/username
```

Output:

admin

Check password:

```
kubectl exec combined-config-secret-pod -- cat /etc/app-secret/password
```

Output:

MyPassword123

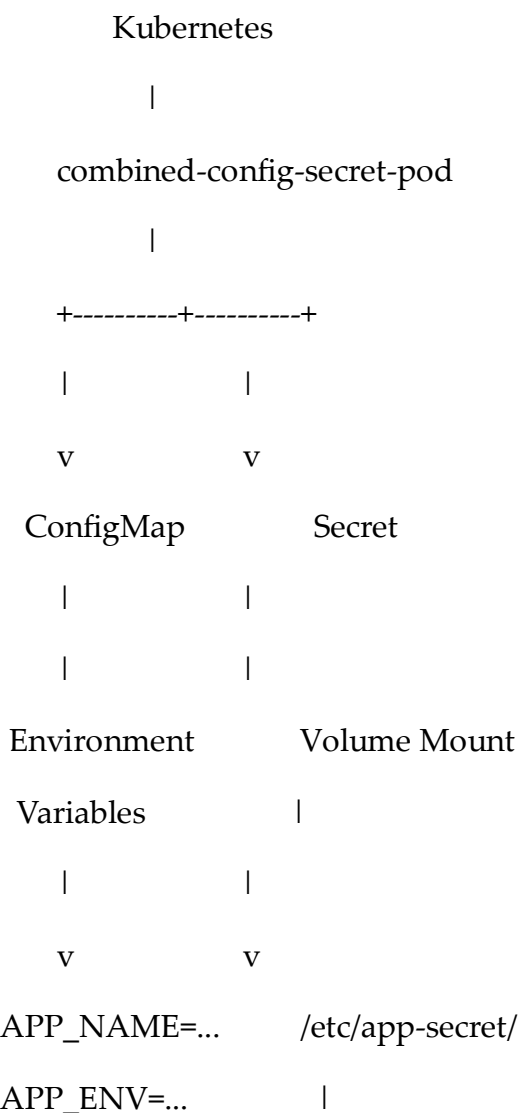
```

ubuntu@k8s-master: ~
ubuntu@k8s-master:~$ kubectl exec combined-config-secret-pod -- ls -l /etc/app-s
ecret
total 0
lrwxrwxrwx 1 root root 15 Sep 28 14:49 password -> ../data/password
lrwxrwxrwx 1 root root 15 Sep 28 14:49 username -> ../data/username
ubuntu@k8s-master:~$ kubectl exec combined-config-secret-pod -- cat /etc/app-sec
ret/username
ubuntu@k8s-master:~$ kubectl exec combined-config-secret-pod -- cat /etc/app-sec
ret/password
MyPassword123ubuntu@k8s-master:~$

```

Final Architecture

Your final Pod looks like this:



```
APP_PORT=...  +-----+-----+
               |           |
               username  password
```

Important Difference to Remember

Feature	ConfigMap	Secret
Purpose	Non-sensitive configuration	Sensitive information
Example	APP_ENV, PORT, URL	Username, password, token
Environment variables	Yes	Yes
Volume mount	Yes	Yes
Base64 required in data YAML	No for ConfigMap	Yes for Secret data
Encryption	Not inherently	Base64 is not encryption; encryption at rest can be configured
Example	APP_ENV=prod	password=...

Useful Commands for Your Lab

See ConfigMaps

kubectl get configmaps

See ConfigMap YAML

kubectl get configmap application-config -o yaml

See Secrets

kubectl get secrets

See Secret YAML

```
kubectl get secret application-secret -o yaml
```

Be careful: this can expose encoded secret values.

Describe Pod

```
kubectl describe pod combined-config-secret-pod
```

Enter the container

```
kubectl exec -it combined-config-secret-pod -- /bin/bash
```

Then:

```
env | grep APP_
```

and:

```
ls /etc/app-secret
```

Cleanup

After completing the lab, you can remove everything:

```
kubectl delete pod configmap-env-pod
```

```
kubectl delete pod configmap-volume-pod
```

```
kubectl delete pod secret-env-pod
```

```
kubectl delete pod secret-volume-pod
```

```
kubectl delete pod combined-config-secret-pod
```

Delete ConfigMaps:

```
kubectl delete configmap app-config
```

```
kubectl delete configmap nginx-config
```

```
kubectl delete configmap application-config
```

Delete Secrets:

```
kubectl delete secret db-secret
```

```
kubectl delete secret database-secret
```

```
kubectl delete secret application-secret
```

The 5 concepts to remember for your seminar

1. ConfigMap → Environment variables

envFrom:

configMapRef:

2. ConfigMap → Files

volumes:

configMap:

3. Secret → Environment variables

valueFrom:

secretKeyRef:

4. Secret → Files

volumes:

secret:

5. Same Pod:

ConfigMap → Environment variables

Secret → Volume/files